6.0 VU Theoretische Informatik (192.017)

Teil 2: Berechenbarkeit (Vertiefung)

Chris Fermüller

Institut für Logic and Computation

Wintersemester 2026

Teil 2 – Berechenbarkeit (Überblick)

Teil 2.1: Die Church-Turing These
Teil 2.2: Imperatives Paradigma: Turingmaschinen und mehr
Teil 2.3: Funktionales Paradigma: λ-Kalkül
Teil 2.4: Rekursive Funktionen
Teil 2.5: Der Satz von Rice und seine Anwendung
Teil 2.6: Weitere unentscheidbare Probleme
Teil 2.7: Unkonventionelle Berechenbarkeitsmodelle

Folgendes wissen wir aus Teil 1:

- Es gibt konkrete unentscheidbare (Entscheidungs-)Probleme: z.B. das Halteproblem (und viele mehr)
- Unentscheidbare Probleme sind oft semi-entscheidbar: Halteproblem, Gültigkeit prädikatenlogischer Formeln, Korrektheit bezogen auf einzelne Input/Output-Paare, Code-Erreichbarkeit, ...
- Viele wichtige Probleme sind nicht einmal semi-entscheidbar: z.B. Programmäquivalenz, universelles Halteproblem, Korrektheit bezogen auf allgemeine Vor- und Nach-Bedingungen ($\rightarrow$ Teil 5), ...
- Ähnliches gilt für Funktionen: Die meisten (der überabzählbar vielen) Funktionen sind unberechenbar ($\rightarrow$ Diagonalverfahren)

Wie robust ist unser Berechenbarkeitsmodell?

Beobachtung: "Entscheidbarkeit" und "Berechenbarkeit" wurde in Teil 1 – über die Sprache SIMPLE – flexibel, aber etwas informell – definiert.
Das ist für viele Zwecke adäquat, lässt aber folgende Frage offen:

- Führen unterschiedliche Modelle zu unterschiedlichen Begriffen von Berechenbarkeit? Oder sind "entscheidbares Problem" und "berechenbare Funktion" unabhängig vom Berechenbarkeitsmodell?

Church-Turing-These (Variante 1 – Turing)

Wenn eine Funktion in irgendeinem konkreten Sinn berechenbar ist, so ist sie auch schon auf einer Turingmaschine berechenbar.

Church-Turing-These (Variante 2 – Turing)

Gibt es ein endlich beschreibbares Verfahren zur exakten Spezifizierung einer formalen Sprache L, so gibt es eine Turingmaschine, die L akzeptiert.

Fragen zur Church-Turing-These

- Wie kommt es zu dieser These?
- Wer war Church und was hat er damit zu tun?
- Wie haben Turing bzw. Church für ihre These argumentiert?
- Wie hat die Fachwelt reagiert?
- Gibt es ernsthafte Versuche die These zu 'beweisen' oder widerlegen?
- Welche Bedeutung hat die These für die Informatik?

Eine wichtige Beobachtung:

Formale Sprachen und Funktionen lassen sich ineinander übersetzen:
– Sprache ($\subseteq \Sigma^*$) $\longrightarrow$ charakteristische Funktion $\Sigma^* \to \{0,1\}$
– Funktionen $\mathbb{N}^k \to \mathbb{N}^m$ oder $(\Sigma^*)^k \to (\Sigma^*)^m$ sind Mengen von Paaren von Tupeln, daher darstellbar als Mengen von Wörtern, also als Sprachen

Auch die Beschränkung auf $\mathbb{N}$ bzw. auf Strings aus $\{0,1\}^*$ ist für die prinzipielle Berechen- oder Entscheidbarkeit unwesentlich.

Ein Problem von David Hilbert

"Wir müssen wissen, und wir werden wissen."



- David Hilbert (1862-1943)
- einer der größten und produktivsten Mathematiker aller Zeiten
- "... Eine Diophantische Gleichung [...] sei vorgelegt: man soll ein Verfahren angeben, nach welchem sich mittelst einer endlichen Anzahl von Operationen entscheiden lässt, ob die Gleichung in ganzen rationalen Zahlen lösbar ist ..."

Ähnlich: Entscheidungsproblem für prädikatenlogische Gültigkeit

Hilbertsches Programm: Formalisierung der gesamten Mathematik um ihre Widerspruchsfreiheit beweisen zu können

David Hilbert: Mathematische Probleme - Vortrag, gehalten auf dem internationalen Mathematiker-Kongreß zu Paris 1900. In: Nachrichten von der Königl. Gesellschaft der Wissenschaften zu Göttingen. Mathematisch-Physikal. Klasse, Heft 3, 1900, 253-297

Was motivierte Church und Turing?

Antwort: Eine negative Lösung zu Hilberts Problemen zu finden!

Beobachtung zu positiven und negativen Instanzen von 'Verfahren':
Positiv: ein informelles Modell der Berechenbarkeit genügt.
Negativ (=Unberechenbarkeitsbeweis): verlangt ein präzises Modell!

Angeregt durch Hilberts Entscheidungsproblem schlug Church 1936 eine mathematische Definition von Berechenbarkeit vor ($\rightarrow$ 2.3: λ-Kalkül).
Problem: nicht intuitiv, (zumindest zunächst:) undurchsichtig.

Turings Antwort: Analysiere zuerst, was ein menschlicher Rechner tut!
Unterscheide dabei unwesentliche und prinzipielle Beschränkungen.

Beobachtung: Nur prinzipielle Beschränkungen sind zunächst für eine brauchbare Definition von Berechenbarkeit relevant. Spezifische (Zeit-,Platz-,Mittel-)Beschränkungen können danach 'eingebaut' werden.
($\rightarrow$ Chomsky-Hierarchie (Teil 3), $\rightarrow$ Komplexitätstheorie (Teil 4))

Erst Turings Analyse (siehe 2.2) hat die Fachwelt überzeugt (Gödel, ...)!

Gibt es ernsthafte Beweis- oder Widerlegungs-Versuche?

Antwort: erstaunlich(?) wenige!
Diagnose: Turing war wohl sehr überzeugend.
Außerdem haben sich Dutzende andere Modelle als äquivalent erwiesen.

Eine interessante Alternative zu Turingmaschinen:
Kolmogorov(-Uspenski)-Maschinen beruhen ebenfalls auf einer grundlegenden Analyse von 'Rechenverfahren' und beschreiben eine Variante von sogenannten Pointer-Maschinen, die nicht auf einem Band, sondern auf bestimmten Graphen-Strukturen arbeiten.
Siehe `https://esolangs.org/wiki/Kolmogorov_machine`

Aus informatischer Sicht wichtiger: Registermaschinen.

Vorschau:
Wir werden im Folgenden nicht nur die Modelle von Turing und Church präsentieren, sondern auch Registermaschinen, sowie einen weiteren wichtigen mathematischen Zugang zu Berechenbarkeit präsentieren.

Relevanz für die (heutige) Informatik

Zwei Aspekte:

(1) Präzise und robuste Modelle der Berechenbarkeit sind eine Voraussetzung für 'Unmöglichkeitsbeweise'.
Solche Beweise sind oft durchaus praktisch relevant, weil sie aufzeigen, wenn, warum und wie bestimmte Einschränkungen von wichtigen Aufgabenstellungen angebracht sind. Das betrifft, z.B., Korrektheitsbeweise, Redundanzerkennung (optimale Compiler,..), Äquivalenz von Spezifikationen, Äquivalenz von Programmen, etc.

(2) Die sehr unterschiedlichen Zugänge von Church und Turing verweisen auf unterschiedliche Paradigmen von Programmiersprachen:
- TM $\rightarrow$ SIMPLE $\rightarrow$ Java, C, C++,...: imperatives Paradigma
- λ-Kalkül $\rightarrow$ ML, Haskell, ...: funktionales (deklaratives) Paradigma

Teil 2 – Berechenbarkeit (Überblick)

Teil 2.1: Die Church-Turing These

Teil 2.2: Imperatives Paradigma: Turingmaschinen und mehr

Teil 2.3: Funktionales Paradigma: λ-Kalkül

Teil 2.4: Rekursive Funktionen

Teil 2.5: Der Satz von Rice und seine Anwendung

Teil 2.6: Weitere unentscheidbare Probleme

Teil 2.7: Unkonventionelle Berechenbarkeitsmodelle

Turings Analyse eines (menschlichen!) Rechners

Turings Frage: Welche reelle Zahlen sind (intuitiv) berechenbar?
(Impliziert auch Antworten für Funktionen und Entscheidungsprobleme!)

Auszüge aus Alan Turing, 'On computable numbers, with an application to the Entscheidungsproblem' (1936):

The arguments which I shall use are of three kinds.
(a) A direct appeal to intuition.
(b) A proof of the equivalence of two definitions. [...]
(c) Giving examples of large classes of numbers which are computable.

Computing is normally done by writing certain symbols on paper.

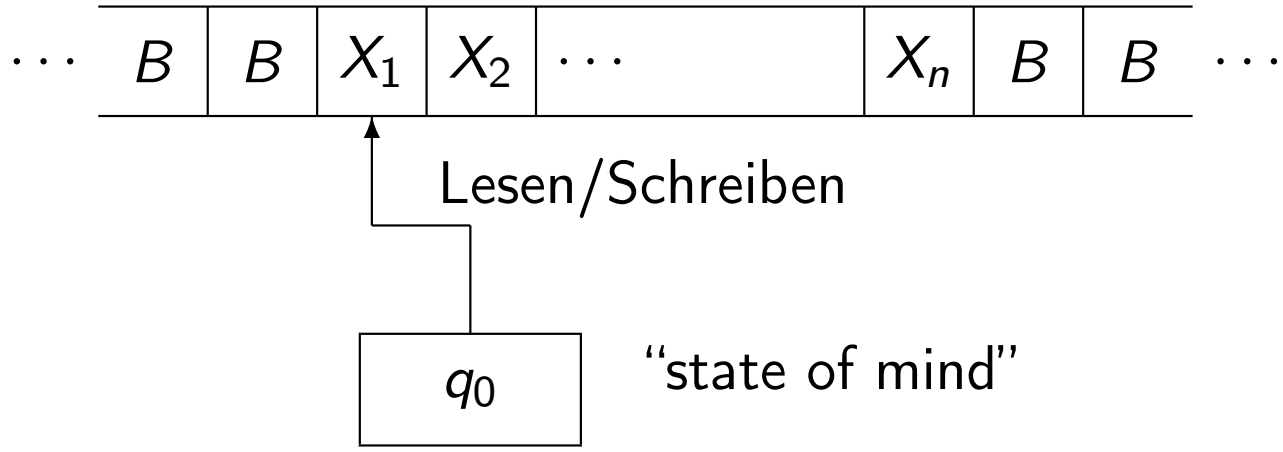
... two-dimensional character of the paper [is] always avoidable ...

... restriction to [finite] number of symbols is not very serious. It is always possible to use sequences of symbols in the place of single symbols. ...

The behaviour of the computer at any moment is determined by the symbols which [s]he is observing, and his "state of mind" at that moment...

We may suppose that in a simple operation not more than one symbol is altered. Any other changes can be split up into simple changes of this kind...

Resultat der Analyse: Turingmaschinen



- Das Band enthält die Eingabe. Alle anderen Zellen enthalten ein Blank-Symbol B. (Nur endliche viele Zellen sind nicht B-gefüllt.)
- Lese/Schreib-Kopf ist immer über einer Bandzelle positioniert.
- Rechenschritt: Ersetze das gelesene Symbol (optional); schiebe den Lese/Schreib-Kopf (möglicherweise) eine Zelle nach links oder rechts; endet in einem (möglicherweise) neuen Zustand ("state of mind").
- Anfang: Kopf ist über der am weitesten links stehenden Eingabezelle.

Formale Beschreibung einer Turingmaschine

Eine Turingmaschine besteht aus 7 Komponenten:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F),$$

- Q: endliche Menge von Zuständen,
- Σ: endliche Menge der Eingabesymbole,
- Γ: endliche Menge der Bandsymbole, $\Sigma \subsetneq \Gamma$,
- Übergangsfunktion $\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, S\}$,
- Startzustand $q_0 \in Q$,
- Blank-Symbol: $B \in \Gamma - \Sigma$,
- Endzustände: $F \subseteq Q$.

Übergänge (= Rechenschritte)

$$(q, X; p, Y, D) \in \delta$$

M liest im Zustand q auf dem Band Symbol X

- wechselt nun davon abhängig in den Zustand p,
- überschreibt Symbol X durch Symbol Y,
- bewegt den Lese/Schreib-Kopf wie durch D beschrieben:
  L... nach links, R... nach rechts, bzw. S... gar nicht.

Wir betrachten im Folgenden nur deterministische Maschinen:

Deterministische Turingmaschine

Eine Turingmaschine M heißt deterministisch, wenn für alle $(q, X) \in Q \times \Gamma$ höchstens ein Element $(q, X; p, Y, D) \in \delta$ existiert,
$D \in \{L, R, S\}$; wir schreiben dann auch $\delta(q, X) = (p, Y, D)$.

M hält/terminiert (in q) wenn $q \in F$ ($\delta(q, X)$ bleibt dann undefiniert.)

Konfiguration / Übergang / Berechnung

Konfiguration: $X_1X_2...X_{i-1}qX_iX_{i+1}...X_n$

TM ist in Zustand q, der Lese/Schreib-Kopf steht über X_i

links von X_1 und rechts von X_n stehen nur Bs

$\Rightarrow$ ... Übergang zwischen Konfigurationen in einem Schritt

$\Rightarrow^*$ ... Übergang zwischen Konfigurationen in beliebig vielen Schritten

- Bewegung nach links, d.h. $\delta(q, X_i) = (p, Y, L)$:

$$X_1X_2...X_{i-1}\boldsymbol{qX_i}X_{i+1}...X_n \Rightarrow X_1X_2...X_{i-2}\boldsymbol{p}X_{i-1}\boldsymbol{Y}X_{i+1}...X_n$$

- Bewegung nach rechts, d.h. $\delta(q, X_i) = (p, Y, R)$:

$$X_1X_2...X_{i-1}\boldsymbol{qX_i}X_{i+1}...X_n \Rightarrow X_1X_2...X_{i-1}\boldsymbol{Yp}X_{i+1}...X_n$$

- Lese/Schreib-Kopf bleibt stehen, d.h., $\delta(q, X_i) = (p, Y, S)$:

$$X_1X_2...X_{i-1}\boldsymbol{qX_i}X_{i+1}...X_n \Rightarrow X_1X_2...X_{i-1}\boldsymbol{pY}X_{i+1}...X_n$$

Akzeptierte Sprache / rekursiv aufzählbar / rekursiv

Akzeptierte Sprache

Die von der Turingmaschine* M akzeptierte Sprache $\mathcal{L}(M)$ besteht aus jenen Wörtern, bei deren Eingabe M terminiert:

$$\mathcal{L}(M) = \{w \in \Sigma^* \mid q_0 w \Rightarrow^* \alpha p \beta, \quad p \in F, \quad \alpha, \beta \in \Gamma^*\}$$

Rekursiv aufzählbare (oder: semi-entscheidbare) Sprachen

L heißt rekursiv aufzählbar wenn sie von einer TM akzeptiert wird.

Rekursive (oder: entscheidbare) Sprachen

L heißt rekursiv (oder entscheidbar), wenn es eine TM gibt, die immer hält und die für jedes Eingabewort $w \in \Sigma^*$ angibt, ob $w \in L$ oder $w \notin L$ (z.B. durch zwei verschiedene entsprechende Endzustände).

Anmerkung: Die Bezeichnungen semi-entscheidbar und entscheidbar beziehen sich auf die Variante 2 der Church-Turing-These.

Berechnung arithmetischer Funktionen ($\mathbb{N}^k \to \mathbb{N}$)

$[n]_2$ ... Binärdarstellung von $n \in \mathbb{N}$

Anfangskonfiguration einer TM M mit $\Sigma = \{0, 1\}$ für $(n_1, ..., n_k)$:
$q_0[n_1]_2 B ... B[n_k]_2$ ... 'M arbeitet auf Eingabe $(n_1, ..., n_k)$'

Endkonfiguration einer (Funktionen-berechnenden) TM M:
$xp[n]_2 By$, $p \in F$, $x, y \in \Gamma^*$, $n \in \mathbb{N}$ ... 'M gibt n aus'

Berechnung von $f : \mathbb{N}^k \to \mathbb{N}$

Eine TM M mit $\Sigma = \{0, 1\}$ berechnet $f : \mathbb{N}^k \to \mathbb{N}$ wenn M auf Eingabe $(n_1, ..., n_k)$ arbeitend, $f(n_1, ..., n_k)$ ausgibt. Schreibweise: $F(M) = f$.

Anmerkung: M terminiert nicht, wenn $f(n_1, ..., n_k)$ undefiniert ist.

Turing-Berechenbarkeit einer arithmetischen Funktion

$f : \mathbb{N}^k \to \mathbb{N}$ heißt Turing-berechenbar, wenn $F(M) = f$ für eine TM M.

$\to$ Church-Turing-These (Variante 1): Berechenbar = Turing-berechenbar

Beispiel

$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_2\})$,

δ gegeben durch:

Zustand	0	1	B
q_0	$(q_0, 0, R)$	$(q_0, 1, R)$	$(q_1, 0, S)$
q_1	$(q_1, 0, L)$	$(q_1, 1, L)$	(q_2, B, R)

Beispielrechnung:

Eingabe: 6, daher auf dem Band $[6]_2 = 110$

$$q_0110 \Rightarrow 1q_010 \Rightarrow 11q_00 \Rightarrow 110q_0[B] \Rightarrow 110q_10$$
$$\Rightarrow 11q_100 \Rightarrow 1q_1100 \Rightarrow q_11100 \Rightarrow q_1B1100 \Rightarrow q_21100$$

Ausgabe: 12, weil auf dem Band $1100 = [12]_2$

Berechnete Funktion: $f_1 = F(M) : \mathbb{N} \to \mathbb{N}$, wobei $f_1(n) = 2n$

Achtung: Dieselbe TM M, angewendet auf Zahlenpaare, berechnet
$f_2 = F(M) : \mathbb{N}^2 \to \mathbb{N}$, wobei $f_2(m, n) = m2^{bl(n)+1} + n$,
dabei ist bl die binäre Länge: $bl(k+1) = \lfloor \log_2(k+1) \rfloor + 1$, $bl(0) = 1$

Universelle Turingmaschinen

Beobachtung:
Eine TM entspricht nicht einem Computer, sondern einem Programm!

Um ein Modell eines Rechners/Computers zu erhalten, benötigen wir eine Maschine, die beliebige Programme (=TMs) simulieren kann.

TM als Input: $\langle M \rangle$ bezeichnet den (Zahlen-)Code einer Turingmaschine M.

Universelle Turingmaschine

Eine TM U heißt universell, wenn für jede TM M und alle $(n_1, ..., n_k) \in \mathbb{N}^k$ gilt: $F(U)(\langle M \rangle, n_1, ..., n_k) = F(M)(n_1, ..., n_k)$.

Wichtig: Die in dieser Formulierung gewählte Einschränkung auf Funktionen-berechnende TMs ist praktisch, aber unwesentlich!

Theorem (Turing, 1936)

Es gibt universelle Turingmaschinen.

Beobachtung: Jede universelle TM entspricht einem Interpreter.

Varianten von Turingmaschinen

Die folgenden Varianten von Turings Modell sind jeweils berechnungsäquivalent, d.h. es werden dieselben arithmetischen Funktionen berechnet bzw. dieselben Sprachen akzeptiert:

TM-Varianten

- Separates Eingabeband (nur lesbar), einseitiges Arbeitsband; auch ein separates Ausgabeband (nur zum Schreiben) wäre möglich
- Mehrband-TM: mehrere (Arbeits-, Eingabe-, Ausgabe-)Bänder, jedes mit einem eigenen Lese/Schreib-Kopf
- Mehrkopf-Bänder: Mehrere Lese/Schreib-Köpfe pro Band
- Binäres oder unäres Eingabealphabet ($\Sigma = \{0, 1\}$ oder $\Sigma = \{1\}$): Berechnungsäquivalenz modulo entsprechender Codierung
- Nichtdeterministische TM: Jede TM lässt sich 'determinisieren'!

Äquivalenzbeweise jeweils durch Reduktion (Simulation)!

Ein anderes imperatives Modell: Registermaschinen

Motivation: Wir suchen ein Modell, dass einerseits ähnlich einfach wie TMs, andererseits aber näher an heutigen Computern ist. Wir fokussieren dabei auf arithmetische Funktionen (Typ $\mathbb{N}^k \to \mathbb{N}$).

Definition einer Registermaschine (RM)

Eine m-Registermaschine (m-RM) besteht aus 3 Komponenten:

$$R = (S, \boldsymbol{O}, \boldsymbol{T})$$

- $S = \mathbb{N}^m$: Speicherzustände (= Inhalt der m Register)
- $\boldsymbol{O} = \{A_i \mid 1 \leq i \leq m\} \cup \{S_i \mid 1 \leq i \leq m\}$: Speicherbefehle
- $\boldsymbol{T} = \{t_i \mid 1 \leq i \leq m\}$: Testbefehle

Wichtig: Anders als bei TMs werden bei RMs Maschinen und Programme getrennt. Erst mit einem RM-Programm kann man rechnen!

Programme für Registermaschinen

Definition eines Registermaschinen-Programms

Ein (m)-RM-Programm besteht aus einer Startmarke $a \in \mathbb{N}$ und einer endlichen Menge aus Anweisungen:

- Funktionsanweisung: (r, f, p) lies 'r: do f then goto p'
- Testanweisung (r, t, p, q) lies 'r: if t then goto p else goto q'

r heißt Kennmarke der Anweisung, p bzw. q Sprungmarken $(r, p, q \in \mathbb{N})$. Sprungmarken, die keine Kennmarken sind, heißen Endmarken.

$R(i)$ ... Inhalt des i-ten Registers $(R(i) \in \mathbb{N})$

Semantik der Befehle:

- A_i: $R(i) := R(i) + 1$
- S_i: $R(i) := R(i) \dot{-} 1$ $[R(i)$ bleibt 0, falls $R(i) = 0]$
- t_i: Ist $R(i) = 0$?

Wir betrachten hier nur deterministische Programme:
Jede Kennmarke kommt nur einmal als solche vor.

Rechnen mit RM-Programmen

Berechnung einer Funktion $f : \mathbb{N}^k \to \mathbb{N}$ auf einer m-RM ($k \leq m$):
Anfangszustand: $R(i) = n_i$ für $1 \leq i \leq k$, $R(i) = 0$ für $k+1 \leq i \leq m$
Endzustand: Endmarke erreicht, $f(n_1, ..., n_k) = R(1)$
Beachte: Eine RM hat im Allgemeinen mehr Register als Eingabezahlen

Konfigurationen:
p:$(n_1, \ldots, n_m)$ ... aktuelle Kennmarke ist p; $R(i) = n_i$ nach Ausführung der vorhergehenden Befehle, bzw. anfangs, falls p die Startmarke ist

Ein Additions-Programm:
$ADD = (0,\ \{(0, t_2, 5, 1),\ (1, S_2, 2),\ (2, A_1, 0)\})$

Beispielrechnung:
$$0{:}(5,2) \Rightarrow 1{:}(5,2) \Rightarrow 2{:}(5,1) \Rightarrow 0{:}(6,1)$$
$$\Rightarrow\ 1{:}(6,1) \Rightarrow 2{:}(6,0) \Rightarrow 0{:}(7,0) \Rightarrow 5{:}(7,0)$$

Ein Substraktions-Programm ($\dot{-}$):
$SUB = (0,\ \{(0, t_2, 5, 1),\ (1, S_2, 2),\ (2, S_1, 0)\})$

Beispielrechnung:
$$0{:}(5,2) \Rightarrow 1{:}(5,2) \Rightarrow 2{:}(5,1) \Rightarrow 0{:}(4,1)$$
$$\Rightarrow 1{:}(4,1) \Rightarrow 2{:}(4,0) \Rightarrow 0{:}(3,0) \Rightarrow 5{:}(3,0)$$

Partielle Funktionen / Codierung

Programm für die (echte) Vorgänger-Funktion:
$PRED = (0,\ \{(0, t_1, 0, 1),\ (1, S_1, 2)\})$

Beispielrechnungen:

$$0{:}(2) \Rightarrow 1{:}(2) \Rightarrow 2{:}(1)$$
$$\text{bzw.}0{:}(0) \Rightarrow 0{:}(0) \Rightarrow 0{:}(0) \Rightarrow \ldots \text{ (terminiert nicht)}$$

Anmerkung: Wie TMs, so können auch RM-Programme auf (Codes von) RM-Programmen (z.B. auch auf sich selbst) angewendet werden.

$\langle P \rangle \in \mathbb{N}$ ... Zahlencode des RM-Programms P

Hinweise zur Codierung als Zahlen:
Basis vieler Codierungen ist die Eindeutigkeit der Primfaktorzerlegung:
$\langle n_1, \ldots, n_m \rangle \rightarrow 2^{n_1} \cdot 3^{n_2} \cdot \ldots \cdot p_m^{n_m}$ ist eindeutig decodierbar!

Auch beliebige Strings über einem Alphabet Σ können so eindeutig als einzelne Zahlen codiert werden, indem man den Zeichen des Alphabets jeweils eine bestimmte Zahl zuordnet.

Wichtige Fakten zu Registermaschinen

- Es gibt universelle RM-Programme P_U: Sei $\langle F \rangle$ der Code eines RM-Programms, das $f : \mathbb{N}^k \to \mathbb{N}$ berechnet, dann berechnet P_U die Funktion $u : \mathbb{N}^{k+1} \to \mathbb{N}$, sodass $u(\langle F \rangle, n_1, ..., n_k) = f(n_1, ..., n_k)$.
- RMs und TMs sind berechnungsäquivalent. Das stützt:

Church-Turing-These (Variante 3 – Shepherdson & Sturgis)

Wenn eine arithmetische Funktion informell berechenbar ist, so ist sie auch schon auf einer Registermaschine berechenbar.

- Eine erstaunliche Tatsache:
  Bereits 2-RMs sind – modulo geeigneter Codierungen von Inputs und Outputs als einzelne Zahlen – berechnungsuniversell!
- RMs kann man ziemlich direkt zu 'realistischen' formalen Modellen moderner Rechner verallgemeinern (z.B., *Random Access Machine*, wo Speicherbereiche explizit adressiert werden müssen).

Übliche (höhere) Programmiersprachen

Frage:
Wie sieht man ein, dass SIMPLE bzw. auch übliche Programmiersprachen, implementiert auf diversen Computerarchitekturen (im Prinzip) nicht mehr berechnen können als TMs oder RMs?

Antwort:
Durch die Tatsache, dass alle Programmiersprachen in eine jeweilige Maschinensprache übersetzbar sein müssen. Auf Maschinen-Ebene lassen sich dann (oft aufwändig) geeignete Reduktionen/Simulationen finden.

(Umgekehrt kann man TMs und RMs in höheren Sprachen simulieren.)

Anmerkungen:

- Jeder Einzelfall unterstützt die Church/Turing-These
- Universelle Maschinen entsprechen Interpretern bzw. Compilern

Beobachtung: Alle bisher betrachten Modelle/Sprache sind imperativ!

Bereits Church hatte ein grundlegend anderes Modell im Sinn...

Teil 2 – Berechenbarkeit (Überblick)

Teil 2.1: Die Church-Turing These

Teil 2.2: Imperatives Paradigma: Turingmaschinen und mehr

Teil 2.3: Funktionales Paradigma: λ-Kalkül

Teil 2.4: Rekursive Funktionen

Teil 2.5: Der Satz von Rice und seine Anwendung

Teil 2.6: Weitere unentscheidbare Probleme

Teil 2.7: Unkonventionelle Berechenbarkeitsmodelle

Alonzo Church – Ein Mann mit einer großen Idee ...

Herausforderung durch Hilberts Entscheidungsproblem:

> *"... propose a definition of effective calculability which [...] correspond[s] satisfactorily to the somewhat vague intuitive notion"*

- Alonzo Church (1903 - 1995)
- US-amerikanischer Mathematiker, Logiker und Philosoph und einer der Begründer der theoretischen Informatik
- Entwicklung des Lambda-Kalküls
- Zahlreiche berühmte Dissertanten: Alan Turing, Michael Rabin, Hartley Rogers, J. Barkley Rosser, Dana Scott, Raymond Smullyan, Stephen C. Kleene, ...



A. Church: An Unsolvable Problem of Elementary Number Theory, American Journal of Mathematics 58 (2), 1936, 345 – 363.

Churchs große Idee: Alles ist eine Funktion!

Nicht nur Rechenvorschriften (Programme), sondern auch Zahlen, Wahrheitswerte, Listen, etc., lassen sich als Funktionsausdrücke darstellen!

Ein Notationsproblem:
'$m + n$' ist zweideutig:
Ist die Zahl $m + n$ gemeint? Oder die (2-stellige) Additions-Funktion?
Mit anderen Worten: Ist die Funktion oder deren Ergebnis gemeint?

Die Lösung: λ-Notation ('Lambda-Notation')
$\lambda m \lambda n.m + n$ bezeichnet die Additions-Funktion als solche.
Entsprechend bezeichnet, z.B., $\lambda m.m + n$ die 1-stellige Funktion '$+n$'.

Anstatt:

"Es sei f die Funktion $x \mapsto x + x^3$. Es sei weiters $a = f(5)$."

können wir nun schreiben:

"$a = (\lambda x.x + x^3)(5)$"

Die kompakteste Programmiersprache der Welt: λ-Terme

Definition von λ-Termen

- Jede Variable ist ein λ-Term ($x, y, z, \ldots, f, g, \ldots, p, q \ldots$).
- Wenn M und N λ-Terme sind, dann ist (MN) ein λ-Term; genannt die Anwendung (oder Applikation) von M auf N.
- Wenn x eine Variable und M ein λ-Term ist, dann ist $(\lambda x.M)$ ein λ-Term. M nennt man den Body der Abstraktion $(\lambda x.M)$.

In Backus-Naur Form:

<lterm> ::= <var> | (<lterm><lterm>) | (λ <var>.<lterm>)
<var> ::= x | y | z | ... | f | g | ... | p | q | ...

Schreibkonventionen:

- Äußere Klammern können weggelassen werden.
- Links-Assoziativität: MNL bedeutet $(MN)L$.
- Der Body einer Abstraktion reicht so weit nach rechts wie möglich: Z.B. steht $\lambda x.MN$ für $\lambda x.(MN)$ – und nicht für $(\lambda x.M)N$.
- Wir schreiben auch $\lambda xyz.M$ statt $\lambda x.\lambda y.\lambda z.M$ (etc.)

Beispiele von λ-Termen (ohne Schreibkonventionen)

(xy), $(x(yz))$, $((xy)z)$, $(\lambda x.x)$, $(\lambda x.y)$, $((\lambda x.x)y)$, $((\lambda x.x)(\lambda x.x))$,
$(\lambda x.(\lambda y.(\lambda z.((fx)z))))$, $(\lambda x.(\lambda y.(\lambda z.((fx)z))))(f(\lambda x.(\lambda f.(fx)))))$

Dieselben Beispiele mit Schreibkonventionen

xy, $x(yz)$, xyz, $\lambda x.x$, $\lambda x.y$, $(\lambda x.x)y$, $(\lambda x.x)\lambda x.x$,
$\lambda xyz.fxz$, $(\lambda x.\lambda y.\lambda z.fxz)(f\,\lambda xf.fx)$

Freie Variablen (FV) und gebundene Variablen (GV)

Definition von $FV(t)$ und $GV(t)$

$FV(x) = \{x\}$ $\qquad$ $GV(x) = \{\}$

$FV(MN) = FV(M) \cup FV(N)$ $\qquad$ $GV(MN) = GV(M) \cup GV(N)$

$FV(\lambda x.M) = FV(M) - \{x\}$ $\qquad$ $GV(\lambda x.M) = GV(M) \cup \{x\}$

Beispiele:

$FV(\lambda x.xy) = \{y\}$, $GV(\lambda x.xy) = \{x\}$,
$FV((\lambda yx.x)x) = \{x\}$, $GV((\lambda yx.x)x) = \{x, y\}$

λ-Terme, in denen alle Variablen gebunden sind, heißen Kombinatoren

Currying – Die Würze des funktionalen Programmierens:

- Haskell B. Curry (1900-1982)
- US-amerikanischer Logiker und Mathematiker
- nach ihm ist die funktionale Programmiersprache HASKELL benannt.



Currying stellt mehrstellige Funktion als iterative Anwendung 1-stelliger da:
$f : A_1 \times A_2 \times ... \times A_n \to B$ wird zu $f_{curried} : A_1 \to (A_2 \to (... A_n \to B)...)$
Beachte: verlangt Prefix-Notation: $m + n \to +(m, n) \to (+_{curried} m)n$

Beobachtung: λ-Terme machen konsequent von Currying Gebrauch

Currying wird nicht nur in HASKELL, sondern in praktisch allen gebräuchlichen Programmiersprachen verwendet:
JavaScript, Java, C++, C#, C, Python, Scheme, Ruby, ...

Currying hat auch Anwendungen in der Linguistik
siehe `https://de.wikipedia.org/wiki/Currying`

Auswerten durch Einsetzen: β-Reduktion

λ-Terme werden durch sukzessive Substitution (=Einsetzen) ausgewertet.

Umbenennung gebundener Variablen (wie bei Quantoren): $\lambda x.x = \lambda y.y$

$M\{y/x\}$ steht für 'gebundene Vorkommen von x in M durch y ersetzt'.

Variablen-Umbenennung heißt auch α-Reduktion: $M \to_\alpha M\{y/x\}$.

Achtung: Dabei darf es nicht zur Bindung bisher freier Variablen oder zu einer Veränderung bestehender Bindungsverhältnisse kommen.

Substitution der freien Vorkommen von x durch N: $M[N/x]$

$x[N/x] = N$

$y[N/x] = y$, wenn $x \neq y$

$(MP)[N/x] = (M[N/x])(P[N/x])$

$(\lambda x.M)[N/x] = \lambda x.M$

$(\lambda y.M)[N/x] = \lambda y.(M[N/x])$, wenn $x \neq y$, $y \notin FV(N)$

$(\lambda y.M)[N/x] = \lambda y'.(M\{y'/y\}[N/x])$, wenn $x \neq y$, $y \in FV(N)$, y' neu

β-Reduktion

Ein (Teil-)Term der Form $(\lambda x.M)N$ in t wird in t durch $M[N/x]$ ersetzt (resultierend in t'). Man schreibt $t \to_\beta t'$ und nennt $(\lambda x.M)N$ einen Redex (reducible expression) in t. Ein λ-Term ohne Redex ist in Normalform.

λ-Terme = Programme / Berechnen = β-Reduzieren

Beispiel:

$$\begin{array}{lll} (\lambda x.y)(\underline{(\lambda z.zz)(\lambda w.w)}) & \to_\beta & (\lambda x.y)(\underline{(\lambda w.w)(\lambda w.w)}) \\ & \to_\beta & \underline{(\lambda x.y)(\lambda w.w)} \\ & \to_\beta & y \end{array}$$

Mit y ist natürlich eine Normalform erreicht.

Auch eine andere Redex-Wahl ist möglich:

$$\underline{(\lambda x.y)((\lambda z.zz)(\lambda w.w))} \quad \to_\beta \quad y$$

Beobachtungen:

- Reduktion kann neue Redexe erzeugen
- Durch Reduktion können Redexe verschwinden
- Anzahl der Schritte hängt von der Reduktions-Strategie ab

Rechenergebnis = Normalform

2 wichtige Fragen:

- (1) Gibt es zu jedem Term eine Normalform?
- (2) Ist die Normalform eines Terms eindeutig?

Antwort auf Frage 1: nein!

$$(\lambda x.xx)(\lambda x.xx) \to_\beta (\lambda x.xx)(\lambda x.xx) \to_\beta (\lambda x.xx)(\lambda x.xx) \to_\beta \ldots$$

Wie zu erwarten war: Nicht jede Berechnung terminiert!

Antwort auf Frage 2: ja!

Theorem (Church-Rosser Theorem – Konfluenz)

Normalformen sind eindeutig: Hat ein λ-Term eine Normalform, so führt jede terminierende Reihenfolge von β-Reduktion auf das selbe Ergebnis (modulo α-Reduktionen, also bis auf Variablen-Umbenennung).

Programmieren im λ-Kalkül (1)

Der (nicht 'das') Lambda-Kalkül formalisiert Reduktion als Regelsystem.

Boolesche Funktionen:
Wir definieren Wahrheitswerte und Wahrheitsfunktionen als Kombinatoren:

Booleans / Konditional

$$\begin{aligned} \mathbf{T} &= \lambda xy.x \\ \mathbf{F} &= \lambda xy.y \\ \mathbf{if_then_else} &= \lambda pxy.pxy \end{aligned}$$

Beispiel:

$$\begin{aligned} \mathbf{if_then_else}\,\mathbf{T}\,a\,b &= \underline{(\lambda pxy.pxy)\mathbf{T}\,a\,b} \\ &\to_\beta \underline{(\lambda xy.\mathbf{T}xy)\,a\,b} \\ &\to_\beta \underline{(\lambda y.\mathbf{T}\,a\,y)\,b} \\ &\to_\beta \mathbf{T}\,a\,b \\ &= \underline{(\lambda xy.x)\,a\,b} \\ &\to_\beta \underline{(\lambda y.a)\,b} \\ &\to_\beta a \end{aligned}$$

Programmieren im λ-Kalkül (2)

Boolesche Funktionen (Forts.):

Konjunktion / Disjunktion / Negation

$$\begin{aligned} \mathbf{and} &= \lambda pq.pqp \\ \mathbf{or} &= \lambda pq.ppq \\ \mathbf{not} &= \lambda pab.pba \end{aligned}$$

Beispiel:

$$\begin{aligned} \mathbf{and}\,\mathbf{T}\,\mathbf{F} &= \underline{(\lambda pq.pqp)\,\mathbf{T}\,\mathbf{F}} \\ &\to_\beta \underline{(\lambda q.\mathbf{T}q\mathbf{T})\,\mathbf{F}} \\ &\to_\beta \mathbf{T}\,\mathbf{F}\,\mathbf{T} \\ &= \underline{(\lambda xy.x)\,\mathbf{F}\,\mathbf{T}} \\ &\to_\beta \underline{(\lambda y.\mathbf{F})\,\mathbf{T}} \\ &\to_\beta \mathbf{F} \end{aligned}$$

Programmieren im λ-Kalkül (3)

Boolesche Funktionen (Forts.):

Zur Erinnerung: $\mathbf{not} = \lambda pab.pba$, $\mathbf{T} = \lambda xy.x$, $\mathbf{F} = \lambda xy.y$

Beispiel:

$$
\begin{array}{rcl}
\mathbf{not}\,\mathbf{T} & = & \underline{(\lambda pab.pba)\,\mathbf{T}} \\
& \to_\beta & \lambda ab.\mathbf{T}ba \\
& = & \lambda ab.\underline{(\lambda xy.x)ba} \\
& \to_\beta & \lambda ab.\underline{(\lambda y.b)a} \\
& \to_\beta & \lambda ab.b \\
& \to_\alpha & \lambda xy.y \quad \text{(Variablen-Umbenennung)} \\
& = & \mathbf{F}
\end{array}
$$

Programmieren im λ-Kalkül (4)

Arithmetische Funktionen:

Zahlendarstellung: Church-Numerale

$$\begin{aligned} 0 &= (\lambda fx.x) \\ 1 &= (\lambda fx.fx) \\ 2 &= (\lambda fx.f(fx)) \\ 3 &= (\lambda fx.f(f(fx))) \\ &\vdots \end{aligned}$$

n ist durch n-fache Anwendung $f^{(n)}$ einer bel. Funktion f repräsentiert

Nachfolger / Addition / Multiplikation

$$\begin{aligned} \mathbf{succ} &= \lambda nfx.f(nfx) \\ \mathbf{add} &= \lambda mnfx.mf(nfx) \\ \mathbf{mult} &= \lambda mnf.m(nf) \end{aligned}$$

add beruht auf der Beobachtung $f^{(m)} \circ f^{(n)} = f^{(m+n)}$

Programmieren im λ-Kalkül (5)

Beispiel:

$$
\begin{aligned}
\mathbf{succ}\,2 &= \underline{(\lambda nfx.f(nfx))\,2} \\
&\to_\beta \lambda fx.f(2fx) \\
&=_\alpha \lambda fx.f(\underline{(\lambda f'x'.f'(f'x'))fx}) \\
&\to_\beta \lambda fx.f(\underline{(\lambda x'.f(fx'))x}) \\
&\to_\beta \lambda fx.f(f(fx)) = 3
\end{aligned}
$$

Beispiel:

$$
\begin{aligned}
\mathbf{add}\,3\,2 &= \underline{(\lambda mnfx.mf(nfx))\,3\,2} \\
&\to_\beta \underline{(\lambda nfx.3f(nfx))\,2} \\
&\to_\beta \lambda fx.3f(2fx) \\
&=_\alpha \lambda fx.3f(\underline{(\lambda f'x'.f'(f'x'))fx}) \\
&\to_\beta \lambda fx.3f\,\underline{((\lambda x'.f(fx'))x)} \\
&\to_\beta \lambda fx.3f(f(fx)) \\
&=_\alpha \lambda fx.\underline{(\lambda f'x'.f'(f'(f'x')))f}\,(f(fx)) \\
&\to_\beta \lambda fx.\underline{(\lambda x'.f(f(fx')))(f(fx))} \\
&\to_\beta \lambda fx.f(f(f(f(fx)))) = 5
\end{aligned}
$$

Programmieren im λ-Kalkül (6)

Beispiel:

$$
\begin{array}{lll}
\textbf{mult}\,2\,3 & = & \underline{(\lambda mnf.m(nf))\,2}\,3 \\
& \to_\beta & \underline{(\lambda nf.2(nf))\,3} \\
& \to_\beta & \lambda f.2(3f) \\
& =_\alpha & \lambda f.2(\underline{(\lambda f'y.f'(f'(f'y)))f}) \\
& \to_\beta & \lambda f.2(\lambda y.f(f(fy))) \\
& =_\alpha & \lambda f.\underline{(\lambda f'z.f'(f'z))}(\lambda y.f(f(fy))) \\
& \to_\beta & \lambda f.(\lambda z.(\lambda y.f(f(fy)))(\underline{(\lambda y.f(f(fy)))z})) \\
& \to_\beta & \lambda f.(\lambda z.\underline{(\lambda y.f(f(fy)))(f(f(fz)))}) \\
& \to_\beta & \lambda f.(\lambda z.f(f(f(f(f(fz)))))) \\
& = & \lambda fz.f(f(f(f(f(fz))))) \\
& =_\alpha & \lambda fx.f(f(f(f(f(fx))))) = 6
\end{array}
$$

Y-Kombinator: Das coolste Programmierkonstrukt der Welt?!

Siehe `https://www.youtube.com/watch?v=BC8ZAMwfwi4`:



Rekursion via Fixpunkt-Kombinator

β-Äquivalenz

Zwei λ-Terme M, M' heißen β-äquivalent ($M =_\beta M'$) wenn M durch $n \geq 0$, eventuell inverse, β-Reduktionen zu M' transformiert werden kann.

Fixpunkt

N heißt Fixpunkt von F wenn $FN =_\beta N$.

Theorem (Fixpunkt-Satz)

Jeder λ-Term hat einen Fixpunkt.

Beweis:

Der Kombinator $\mathbf{Y} = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$ ist ein Fixpunkt-Generator: Es gilt $\mathbf{Y}N =_\beta N\mathbf{Y}N$ für jeden λ-Term N.

Bedeutung: $\mathbf{Y}$ erlaubt die Darstellung beliebiger Rekursionen!

Für mehr Details siehe, z.B., `https://arxiv.org/abs/0804.3434`

Äquivalenz von Turingmaschinen und λ-Kalkül

Church-Numerale erlauben es, arithmetische Funktionen, d.h. Funktionen vom Typ $\mathbb{N}^k \to \mathbb{N}$ als λ-Terme zu codieren.
Church schuf sein Berechnungsmodell einige Monate vor Turing (1936).

Church-Turing-These (Variante 4 – Church)

Wenn eine arithmetische Funktion informell berechenbar ist, dann lässt sie sich als λ-Term darstellen und im λ-Kalkül berechnen (= λ-definierbar).

Ein wichtiger Schritt zur Erhärtung der These:

Theorem (Turing, 1936)

Eine arithmetische Funktion ist genau dann λ-definierbar, wenn sie Turing-berechenbar ist.

Damit ist von zumindest zwei sehr unterschiedlichen Programmier-Paradigmen gezeigt, dass sie prinzipiell gleich ausdrucksstark sind.

Teil 2 – Berechenbarkeit (Überblick)

Teil 2.1: Die Church-Turing These

Teil 2.2: Imperatives Paradigma: Turingmaschinen und mehr

Teil 2.3: Funktionales Paradigma: λ-Kalkül

Teil 2.4: Rekursive Funktionen

Teil 2.5: Der Satz von Rice und seine Anwendung

Teil 2.6: Weitere unentscheidbare Probleme

Teil 2.7: Unkonventionelle Berechenbarkeitsmodelle

Kurt Gödel – Ein Logiker mit großer Folgewirkung

"Either mathematics is too big for the human mind or the human mind is more than a machine."

- Kurt Gödel (1906 - 1978)
- österreichisch-amerikanischer Logiker
- berühmt (u.a.) für den Beweis der Vollständigkeit der Prädikatenlogik (1929) und seine Unvollständigkeitssätze (1931), die das Hilbertsche Programm unterminieren
- wurde vom TIME Magazine zu einer der 100 bedeutendsten Personen des 20. Jh. erklärt



Kurt Gödel: über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. in: Monatshefte für Mathematik und Physik 38, 1931, 173 – 198.

Rózsa Péter – Pionierin der Theorie rekursiver Funktionen

"When I began my college education, I still had many doubts about whether I was good enough for mathematics. Then a colleague said the decisive words to me: it is not that I am worthy to occupy myself with mathematics, but rather that mathematics is worthy for one to occupy oneself with."

- Rózsa Péter (1905 - 1977)
- ungarische Mathematikerin und Logikerin
- Begründerin der Theorie rekursiver Funktionen
- inspiriert von Gödels Unvollständigkeitssätzen entwickelte sie eine eigenen Zugang zu Berechenbarkeit und Unentscheidbarkeit



Rózsa Péter: Rekursive Funktionen, Budapest 1951. (Monographie, mehrere Übersetzungen und weitere Auflagen)

Ein klassischer Zugang zu Berechenbarkeit

Grundidee (induktive Charakterisierung von Berechenbarkeit):
Man fängt mit einigen offensichtlich berechenbaren Grundfunktionen an und definiert möglichst wenige, schlanke Operatoren (Funktionalen), die gegebene Funktionen in ebenfalls berechenbare Funktionen abbilden.

Wir betrachten (o.B.d.A.!) Funktionen $\mathbb{N}^k \to \mathbb{N}$, $k \geq 0$

Grundfunktionen

- konstante Funktionen C_n^k: $C_n^k(x_1, ..., x_k) = n$
- Nachfolger-Funktion S: $S(x) = x + 1$
- Projektions-Funktionen P_i^k: $P_i^k(x_1, ..., x_k) = x_i$ $(1 \leq i \leq k)$

Beispiele:

C_2^1 $[f(x) = 2, \lambda x.2]$ C_0^3 $[f(x, y, z) = 0, \lambda xyz.0]$ C_2^0 $[2]$ (C_3^0, C_4^0) $[(3, 4)]$
S $[f(x) = x + 1, \mathbf{succ}]$ (S, C_2^1) $[f(x) = (x + 1, 2)]$
P_1^3 $[f(x, y, z) = x, \lambda xyz.x]$ P_3^3 $[f(x, y, z) = z, \lambda xyz.z]$ P_1^1 $[f(x), \lambda x.x]$
(P_2^2, P_1^2) $[f(x, y) = (y, x)]$ (P_3^3, P_2^3, P_2^3) $[f(x, y, z) = (z, y, y)]$

Einfache Zusammensetzung von Funktionen

Komposition $\circ$ (auch: Einsetzungs-Operator)

Gegeben $h : \mathbb{N}^m \to \mathbb{N}$ und für $1 \leq i \leq m$: $g_i : \mathbb{N}^k \to \mathbb{N}$ $(k \geq 0)$, ist

$$f = (h \circ (g_1, ..., g_m)) : \mathbb{N}^k \to \mathbb{N}$$

definiert durch $f(x_1, ..., x_k) = h(g_1(x_1, ..., x_k), \ldots g_m(x_1, ..., x_k))$.

Links-Assoziativität: $A \circ B \circ C = (A \circ B) \circ C$ (also wie bei λ-Termen)

Beispiele:

$S \circ P_2^2$ $[f(x, y) = y + 1]$ $\quad$ $C_0^1 \circ S$ $[f(x) = 0]$ $\quad$ $S \circ C_0^1$ $[f(x) = 1]$

$S \circ S \circ S$ $[f(n) = n + 3]$ $\quad$ $S \circ S \circ C_0^0$ $[2]$

$P_2^2 \circ (C_1^1, S)$ $[f(x) = x + 1]$

$$\begin{aligned} P_2^2 \circ (S \circ S, S) \circ (S \circ C_0^1) \circ P_1^1 &= (P_2^2 \circ (S \circ S, S)) \circ (S \circ C_0^1) \\ &= (P_2^2 \circ (S \circ S, S)) \circ C_1^1 \\ &= S \circ C_1^1 = C_2^1 \end{aligned}$$

Beachte: Die Sprache der Funktionsausdrücke ist variablenfrei!

Einfache ('primitive') Rekursion

Beobachtung: Eine induktive (= rekursive) Definition einer Funktion bezieht sich auf 2 Bestandteile: Anfangsfunktion und Übergangsfunktion.

Beispiel: rekursive Definition von Addition

Anfang: $Add(0, n) = n$ $[0 + n = n]$

Übergang: $Add(S(m), n) = S(Add(m, n))$ $[(m + 1) + n = (m + n) + 1]$

Übergangsfunktion kann sich auch auf das Argument n beziehen:

Beispiel: rekursive Definition von Multiplikation

Anfang: $Mult(0, n) = 0$ $[0 \cdot n = 0]$

Übergang: $Mult(S(m), n) = Add(n, Mult(m, n))$ $[(m + 1) \cdot n = m \cdot n + n]$

Übergangsfunktion kann sich auch auf die Rekursionstiefe m beziehen:

Beispiel: Die Faktorielle (auch: Fakultäts-Funktion)

Anfang: $Fact(0) = 1$ $[0! = 1]$

Übergang: $Fact(S(m)) = Mult(S(m), Fact(m))$ $[(m + 1)! = (m + 1) \cdot m!]$

Der Operator **Pr** der primitiven Rekursion

Primitive Rekursion **Pr**

Gegeben $g : \mathbb{N}^k \to \mathbb{N}$ und $h : \mathbb{N}^{k+2} \to \mathbb{N}$ $(k \geq 0)$, ist

$$f = \mathbf{Pr}(g, h) : \mathbb{N}^{k+1} \to \mathbb{N}$$

definiert durch $f(0, x_1, ..., x_k) = g(x_1, ..., x_k)$ und
$f(m+1, x_1, ..., x_k) = h(m, f(m, x_1, ..., x_k), x_1, ..., x_k)$.

Die mit **Pr** und $\circ$ aus Grundfunktionen definierbaren Funktionen heißen primitiv-rekursive Funktionen.

Beispiel: $Add = \mathbf{Pr}(P_1^1, S \circ P_2^3)$

$$\begin{aligned} Add(0,5) &= P_1^1(5) = 5 \\ Add(1,5) &= S \circ P_2^3(0,5,5) = S(5) = 6 \\ Add(2,5) &= S \circ P_2^3(1,6,5) = S(6) = 7 \\ Add(3,5) &= S \circ P_2^3(2,7,5) = S(7) = 8 \end{aligned}$$

Primitive Rekursion – weitere Beispiele

Multiplikation:

$Mult = \mathbf{Pr}(C_0^1, Add \circ (P_2^3, P_3^3)) = \mathbf{Pr}(C_0^1, \mathbf{Pr}(P_1^1, S \circ P_2^3) \circ (P_2^3, P_3^3))$
$Mult(m, n)$: C_0^1 bezieht sich auf n, $Add \circ (P_2^3, P_3^3)$ auf $(m, Mult(m, n), n)$

Faktorielle:

$Fact = \mathbf{Pr}(C_1^0, Mult \circ (P_2^2, S \circ P_1^2)) = \mathbf{Pr}(C_1^0, \mathbf{Pr}(C_0^1, \mathbf{Pr}(P_1^1, S \circ P_2^3 \ldots$
$Fact(m)$: C_1^0 hat kein Argument (daher ist der obere Index 0)
$Mult \circ (P_2^2, S \circ P_1^2)$ bezieht sich auf $(m, Fact(m))$:
$Mult \circ (P_2^2, S \circ P_1^2)(m, Fact(m)) = Mult \circ (Fact(m), m+1) = Fact(m) \cdot (m+1)$

Abgeschnittene Vorgängerfunktion:

$APred = \mathbf{Pr}(C_0^0, P_1^2)$, weil $APred(0) = 0$ und $APred(m+1) = m$
$APred(5) = \mathbf{Pr}(C_0^0, P_1^2)(4+1) = P_1^2(4, APred(4)) = 4$

Charakteristische Funktion für '$\neq 0$' (Signum):

$Signum = \mathbf{Pr}(C_0^0, C_1^2)$, weil $Signum(0) = 0$ und $Signum(m+1) = 1$

Abgeschnittene Subtraktion:

$ASub = \mathbf{Pr}(P_1^1, APred \circ P_2^3) \circ (P_2^2, P_1^2)$ $\quad ASub(m, n) = \begin{cases} m - n & \text{falls } m \geq n \\ 0 & \text{sonst} \end{cases}$

Boolesche Funktionen / Fallunterscheidung

Wir verwenden die übliche Codierung: 1 für **wahr** / 0 für **falsch**

Negation / Konjunktion / Disjunktion:

$Not = Asub \circ (C_1^1, P_1^1)$ $[Not(x) = Asub(1, x)]$

$And = Mult$ $[And(x, y) = 1 \iff x \cdot y = 1,\ 0 \text{ sonst}]$

$Or = Signum \circ Add$ $[Or(x, y) = 1 \iff x + y > 0,\ 1 \text{ sonst}]$

Charakteristische Funktion $\chi(<)$ für die 'kleiner'-Relation:

$less = Signum \circ ASub \circ (P_2^2, P_1^2)$ $[less(m, n) = 1 \iff m < n,\ 0 \text{ sonst}]$

Beachte: Alle Booleschen Ausdrücke über Ungleichungen sind ausdrückbar!

Definition durch Fallunterscheidung:

$$f(m, n) = \begin{cases} g(m, n) & \text{falls } m < n \\ h(m, n) & \text{sonst} \end{cases}$$

G bzw. H seien Funktionsausdrücke für g bzw. h. Dann ist

$$Add \circ (Mult \circ (G, less),\ Mult \circ (H, Not \circ less))$$

ein Funktionsausdruck für $f = g(m, n) \cdot \chi(m < n) + h(m, n) \cdot \chi(m \geq n)$

Minimierung – μ_0- und $\overline{\mu}$-Operator

Beobachtung: Primitiv rekursive Funktionen sind total (überall definiert).
Für, z.B., echte Subtraktion (in $\mathbb{N}$) benötigen wir einen weiteren Operator.

μ_0- und $\overline{\mu}$-Rekursion

Gegeben eine totale Funktion $g : \mathbb{N}^{k+1} \to \mathbb{N}$ $(k \geq 0)$, sind

$$f = \mu_0 g : \mathbb{N}^k \to \mathbb{N} \quad \text{und} \quad f' = \overline{\mu} g : \mathbb{N}^{k+1} \to \mathbb{N}$$

definiert durch $f(x_1, ..., x_k) = \min_{y \geq 0}[g(y, x_1, ..., x_k) = 0]$ bzw.
durch $f'(x_1, x_2, ..., x_{k+1}) = \min_{y \geq 0}[g(y, x_2, ..., x_{k+1}) = x_1]$.

Beachte: Die minimale Zahl muss jeweils aus $\mathbb{N}$ sein!

$(\mu_0 h)(x_1, ..., x_k) = (\overline{\mu} h)(0, x_1, x_2, ..., x_k)$
Umgekehrt lässt sich $\overline{\mu} f$ mit Hilfe von μ_0, **Pr** und $\circ$ darstellen.

Die mit μ_0 (äquivalent: $\overline{\mu}$), **Pr** und $\circ$ aus Grundfunktionen definierbaren Funktionen heißen μ-rekursive oder auch partiell rekursive Funktionen.
(Das entsprechende Minimum muss nicht existieren, daher 'partiell'.)
Wenn eine solche Funktion überall definiert ist, heißt sie total rekursiv.

Minimierung – Beispiele

Beispiele

$(\mu_0 P_1^1)(n) = \min_{y \geq 0}[y{=}0] = 0$, daher $\mu_0 P_1^1 = C_0^0$

$(\overline{\mu} P_1^1)(n) = \min_{y \geq 0}[y{=}n] = n$, daher $\overline{\mu} P_1^1 = P_1^1$

$(\mu_0 S)(n) = \min_{y \geq 0}[y + 1 = 0]$ ist immer undefiniert; so wie $\mu_0 C_k^1$ für $k \geq 1$

$(\overline{\mu} S)(n) = \min_{y \geq 0}[y + 1 = n]$ ist die (echte) Vorgänger-Funktion,
also undefiniert für $n = 0$

$(\overline{\mu} Add)(m, n) = \min_{y \geq 0}[y + n = m] = m - n$, undefiniert für $m < n$
also ist $Sub = \overline{\mu} Add$ die (echte) Subtraktion in $\mathbb{N}$

$(\overline{\mu} Mult)(m, n) = \min_{y \geq 0}[y \cdot n = m] = \frac{m}{n}$, falls m durch n teilbar ist
und undefiniert sonst
also ist $Div = \overline{\mu} Mult$ die (echte) Division in $\mathbb{N}$

Partielle Rekursivität und Berechenbarkeit

Beobachtung:
Jede primitiv rekursive Funktion ist berechenbar; z.B., mit SIMPLE.
Auch jede partiell rekursive Funktion ist berechenbar, da μ und $\overline{\mu}$ nur auf totale (berechenbare) Funktionen angewendet werden. Die Suche nach der minimalen Lösung terminiert aber nicht immer.

Umgekehrt gilt auch:

Theorem (Turing, Kleene, Péter)

Jede auf einer TM berechenbare Funktion ist partiell rekursiv.

Daraus ergibt sich:

Church-Turing-These (Variante 5 – Gödel/Kleene/Church/Péter)

Die Klasse der informell berechenbaren Funktionen stimmt mit der Klasse der partiell rekursiven Funktionen überein.

Berechenbar und total, aber nicht primitiv rekursiv

Beobachtungen:

- Die Funktionsausdrücke, die durch Anwendung von **Pr** und $\circ$ auf Grundfunktionen entstehen lassen sich systematisch aufzählen. F_0, F_1, F_2, ... seien die entsprechenden Funktionen vom Typ $\mathbb{N} \to \mathbb{N}$.
- Alle Einträge in folgender (unendlichen) Tabelle sind berechenbar:

	F_0	F_1	F_2	F_3	...
0	$F_0(0)$	$F_1(0)$	$F_2(0)$	$F_3(0)$	...
1	$F_0(1)$	$F_1(1)$	$F_2(1)$	$F_3(1)$	...
2	$F_0(2)$	$F_1(2)$	$F_2(2)$	$F_3(2)$	...
3	$F_0(3)$	$F_1(3)$	$F_2(3)$	$F_3(3)$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

- Cantor'sches Diagonalverfahren: Die Funktion $D(n) = F_n(n) + 1$ ist berechenbar, kann aber für kein i mit F_i übereinstimmen.

Konsequenzen für die Informatik

Aus unseren Beobachtungen ergibt sich:

Theorem

Nicht alle totalen berechenbaren Funktionen sind primitiv rekursiv sind.

Allgemeiner folgt (via Church-Turing-These):

Garantierte Termination erzwingt Unvollständigkeit

Es kann keine Programmiersprache geben, in der alle Programme terminieren und in der alle totalen, berechenbaren Funktionen ausdrückbar sind.

Anmerkungen:

- Man benützt hier, dass sich Programme immer systematisch aufzählen lassen müssen. Sonst könnte man für diese Sprache auch keinen Parser (Compiler / Interpreter) schreiben.
- Es gibt Programmiersprachen (bzw. Umgebungen), in denen nur terminierende Programme formuliert werden können. Darin ist aber eben nicht jeder terminierende Algorithmus ausdrückbar.

Eine total rekursive, aber nicht primitiv rekursive Funktion

Ackermann-Funktion

Die Funktion $Ack : \mathbb{N}^2 \to \mathbb{N}$ ist durch folgende Gleichungen definiert:

$$\begin{aligned} Ack(0, n) &= n + 1 \\ Ack(m + 1, 0) &= Ack(m, 1) \\ Ack(m + 1, n + 1) &= Ack(m, Ack(m + 1, n)) \end{aligned}$$

Es ist nicht ganz offensichtlich, dass Ack berechenbar ist!
Übungsaufgabe: Berechnen Sie $Ack(3, 0)$!

Es gilt:

$$\begin{aligned} Ack(1, n) &= n + 2 \\ Ack(2, n) &= 2n + 3 \\ Ack(3, n) &= 2^{n+3} - 3 \\ Ack(4, n) &= \underbrace{2^{2^{\cdots^2}}}_{n+3 \text{ Zweier}} - 3 \end{aligned}$$

Theorem

Ack ist berechenbar und total, aber nicht primitiv rekursiv.

Teil 2 – Berechenbarkeit (Überblick)

Teil 2.1: Die Church-Turing These
Teil 2.2: Imperatives Paradigma: Turingmaschinen und mehr
Teil 2.3: Funktionales Paradigma: λ-Kalkül
Teil 2.4: Rekursive Funktionen
Teil 2.5: Der Satz von Rice und seine Anwendung
Teil 2.6: Weitere unentscheidbare Probleme
Teil 2.7: Unkonventionelle Berechenbarkeitsmodelle

Noch einmal: Entscheidungsprobleme

Zur Erinnerung:
Wir identifizieren (Entscheidungs)probleme mit Sprachen ($\subseteq \Sigma^*$)

Beispiele: Menge der haltenden Programme, gültigen PL-Formeln, etc.
Gegeben ein beliebiges Objekt s (z.B. ein Programm oder eine PL-Formel):
Gilt $s \in X$?

(Un)entscheidbar heißt:
Es gibt (k)ein haltendes Programm, das $s \in X$? immer korrekt beantwortet.

Wir haben bisher einzelne Probleme X *case-by-case* als unentscheidbar erkannt. (Wenn X und somit auch $\overline{X}$ unentscheidbar sind, dann ist zumindest eines der Probleme X bzw. $\overline{X}$ nicht einmal semi-entscheidbar.)
Beweismethode: Reduktion eines bereits als unentscheidbar erkannten Problems (typischerweise des Halteproblems) auf X.

Herausforderung:
Finde eine Methode, die es erlaubt ganze Klassen von Problemen als unentscheidbar zu erkennen, ohne neue Reduktionen erfinden zu müssen.

Eigenschaften von Sprachen

Wie bereits mehrfach betont, müssen 2 Ebenen strikt getrennt bleiben:

(1): Sprachen (= Probleme), Funktionen, (Input/Output-)Relationen,...
(2): Grammatiken, Programme, Turingmaschinen (TMs), λ-Terme,...

Ebene 2 bilden Darstellungsmittel, Ebene 1 die dargestellten Objekte.
Entscheidungsprobleme beziehen sich auf Ebene 1 und fragen, ob es entsprechende Entscheidungsmechanismen der Ebene 2 gibt.

Wir konzentrieren uns zunächst in der Ebene 1 auf Sprachen $\subseteq \Sigma^*$.

Um die **Herausforderung** annehmen zu können, benötigen wir:

(Extensionale) Eigenschaften rekursiv aufzählbarer Sprachen

Eine (Sprach-)Eigenschaft ist eine Teilmenge $\mathcal{E}$ der rekursiv aufzählbaren Sprachen über einem geeigneten Alphabet Σ. (Also $\mathcal{E} \subsetneq \mathcal{P}(\Sigma^*)$.)
$\mathcal{E}$ heißt trivial, wenn sie entweder leer ist oder aus allen rekursiv aufzählbaren Sprachen (über Σ) besteht.

Beispiele von Sprach-Eigenschaften

Beispiele für Eigenschaften rekursiv aufzählbarer Sprachen

$\mathcal{E}_1 = \{L \mid L \text{ ist entscheidbar } \}$
$\mathcal{E}_2 = \{L \mid L \text{ ist semi-entscheidbar } \}$
$\mathcal{E}_3 = \{L \mid L \text{ ist endlich } \}$
$\mathcal{E}_4 = \{L \mid L = L^*\}$
$\mathcal{E}_5 = \{\{\varepsilon\}\}$
$\mathcal{E}_6 = \{\}$
$\mathcal{E}_7 = \{\{\}\}$
$\mathcal{E}_8 = \{L \mid L = \{a, b\}^* \text{ oder } L = \{a\}^*\}$

Nur $\mathcal{E}_2$ und $\mathcal{E}_6$ sind triviale Eigenschaften.

Beispiele für Klassifikationen

$\{a\}^*$ hat die Eigenschaften $\mathcal{E}_1, \mathcal{E}_2, \mathcal{E}_4, \mathcal{E}_8$, aber nicht $\mathcal{E}_3, \mathcal{E}_5, \mathcal{E}_6, \mathcal{E}_7$.
$\{\varepsilon, a, aa\}$ hat die Eigenschaften $\mathcal{E}_1, \mathcal{E}_2, \mathcal{E}_3$, aber nicht $\mathcal{E}_4, \mathcal{E}_5, \mathcal{E}_6, \mathcal{E}_7, \mathcal{E}_8$.
$\{\langle M \rangle \mid \langle M \rangle \text{ ist der Code einer TM } M\}$ hat nur die Eigenschaften $\mathcal{E}_1, \mathcal{E}_2$.

Der Satz von Rice† – Ein mächtiges Werkzeug

Theorem (Satz von Rice – informelle Kurzform)

Jede nicht-triviale Sprach-Eigenschaft ist unentscheidbar.

Beachte: Genau genommen können nur Sprachen (un)entscheidbar sein. Eigenschaften sind aber Mengen von Sprachen. Diese können indirekt, über ihre jeweilige Darstellung als TM, als Sprachen repräsentiert werden.

Definition (Sprach-Code einer Eigenschaft)

Für jede Eigenschaft $\mathcal{E}$ rekursiv aufzählbarer Sprachen sei $L_{\mathcal{E}} = \{\langle M \rangle \mid \mathcal{L}(M) \in \mathcal{E}\}$, also die Menge aller Codes von jenen TMs, die eine Sprache $L \in \mathcal{E}$ akzeptieren.

(Un-)Entscheidbarkeit von $\mathcal{E}$ meint (Un-)Entscheidbarkeit von $L_{\mathcal{E}}$.

Theorem (Satz von Rice – genauere Formulierung)

Für jede nicht-triviale Eigenschaft $\mathcal{E}$ rekursiv aufzählbarer Sprachen ist $L_{\mathcal{E}}$ unentscheidbar. (Kurz: '$\mathcal{E}$ ist unentscheidbar'.)

Der Satz von Rice: Anwendungen

Einige Beispiele von Eigenschaften, für die der Satz von Rice unmittelbar die Unentscheidbarkeit impliziert.

- Es ist nicht entscheidbar, ob eine gegebene TM mehr als fünf Wörter akzeptiert. (Sprach-Eigenschaft $\mathcal{E} = \{L \mid |L| > 5\}$).
  Um den Satz von Rice anwenden zu können, müssen wir noch überprüfen, ob die Eigenschaft $\mathcal{E}$ nicht-trivial ist. Wir müssen also mindestens eine rekursiv aufzählbare Sprache finden, die die Eigenschaft erfüllt, und eine, die die Eigenschaft nicht erfüllt: z.B. $L_1 = \{a, a^2, a^3, \ldots, a^6\} \in \mathcal{E}$ und $L_2 = \{\} \notin \mathcal{E}$.
- Es ist nicht entscheidbar, ob eine gegebene TM nur endlich viele Wörter akzeptiert. (Sprach-Eigenschaft $\mathcal{E} = \{L \mid L \text{ ist endlich } \}$)
  Kurz: Es ist nicht entscheidbar, ob eine Sprache endlich ist.
- Es ist nicht entscheidbar, ob eine TM überhaupt etwas akzeptiert. (Auch die Spracheigenschaft $\{\{\}\}$ bzw. dessen Komplement $\{L \mid L \neq \{\}\}$ ist nicht-trivial!)

Nicht-Anwendbarkeit des Satzes von Rice – Intensional versus Extensional

Es gibt durchaus entscheidbare Eigenschaften für Turingmaschinen:
Z.B. ist es entscheidbar, ob eine TM mindestens 5 Zustände hat.
Das ist keine Eigenschaft der akzeptierten Sprache, sondern der TM selbst.

Intensionale Eigenschaften:
beziehen sich auf die Darstellungsmittel, hier TMs (→ Folie 62, Ebene 2)

Extensionale Eigenschaften:
beziehen sich auf das Dargestellte, hier Sprachen (→ Folie 62, Ebene 1)

Beachte: Sprach-Eigenschaften sind extensional. Der Satz von Rice bezieht sich also nur auf extensionale, nicht auf intensionale Eigenschaften!

Auch intensionale Eigenschaften können unentscheidbar sein.
Z.B.: Akzeptiert eine vorgelegte TM ihren eigenen Code?

Der Satz von Rice für Funktionen

Nicht nur Eigenschaften von Sprachen, sondern auch Eigenschaften berechenbarer Funktionen sind relevant.

(Extensionale) Eigenschaften berechenbarer Funktionen

Eine (extensionale) Funktions-Eigenschaft ist eine Teilmenge $\mathcal{F}$ der Turing-berechenbaren Funktionen (eines bestimmten Typs).
$\mathcal{F}$ heißt trivial, wenn sie entweder leer ist oder aus allen Turing-berechenbaren Funktionen (des entsprechenden Typs) besteht.

Definition (Sprach-Code einer Funktions-Eigenschaft)

Für jede Funktions-Eigenschaft $\mathcal{F}$ sei $L_{\mathcal{F}} = \{\langle M \rangle \mid F(M) \in \mathcal{F}\}$, also die Menge aller Codes von TMs, die eine Funktion $f \in \mathcal{F}$ berechnen.

Theorem (Satz von Rice für Funktionen)

Für jede nicht-triviale Eigenschaft $\mathcal{F}$ von Turing-berechenbaren Funktionen ist die Sprache $L_{\mathcal{F}}$ unentscheidbar. (Kurz: '$\mathcal{F}$ ist unentscheidbar'.)

Anwendungen des Satzes von Rice für Funktionen

Beispiele:

- Es ist nicht entscheidbar, ob eine Turingmaschine M die Nachfolger-Funktion berechnet.
  (Die Funktions-Eigenschaft $\{f : \mathbb{N} \to \mathbb{N} \mid f(x) = x + 1\}$ ist nicht-trivial.)
- Es ist nicht entscheidbar, ob eine Turingmaschine nur totale arithmetische Funktionen berechnet. (Totalität ist eine nicht-triviale Eigenschaft, weil es totale Funktion gibt die Turing-berechenbar sind, aber nicht alle Turing-berechenbare Funktionen total sind.)
- Es ist nicht entscheidbar, ob für die Funktion $F(M) : \{0,1\}^* \to \{0,1\}^*$ einer Turingmaschine M gilt, dass $F(M)(1) = 111$ oder $F(M)(1) = 000$. (Die Funktions-Eigenschaft $\{f : \{0,1\}^* \to \{0,1\}^* \mid f(1) = 111 \text{ oder } f(1) = 000\}$ ist offensichtlich nicht-trivial.)

Weiteres zur Anwendbarkeit des Satzes von Rice

Beobachtung:
Die Unentscheidbarkeit des Korrektheitsproblems aus Teil 1, angewendet auf arithmetische Funktionen, folgt unmittelbar aus dem Satz von Rice. Das gilt auch für die meisten verwandten Probleme: Z.B. das universelle Halteproblem oder das Äquivalenzproblem.

Anmerkung:
Die Beschränkung auf Turingmaschinen als Darstellungsmittel für Sprachen bzw. Funktionen ist unwesentlich. Analoge Sätze gelten auch für übliche Programmiersprachen, Typ-0-Grammatiken, λ-Terme, etc.

Achtung: Es ist nicht immer einfach festzustellen, ob eine Funktions- oder Sprach-Eigenschaft trivial ist. Z.B. ist es trivial entscheidbar, ob eine von einer TM berechnete Funktion (vom Typ $\mathbb{N}^k \to \mathbb{N}$) λ-definierbar ist.

Die Antwort ist immer 'ja': Jede Turing-berechenbare Funktion ist λ-definierbar. Aber das beruht auf tiefen, keineswegs trivialen Erkenntnissen von Turing und Kleene!

Teil 2 – Berechenbarkeit (Überblick)

Teil 2.1: Die Church-Turing These
Teil 2.2: Imperatives Paradigma: Turingmaschinen und mehr
Teil 2.3: Funktionales Paradigma: λ-Kalkül
Teil 2.4: Rekursive Funktionen
Teil 2.5: Der Satz von Rice und seine Anwendung
Teil 2.6: Weitere unentscheidbare Probleme
Teil 2.7: Unkonventionelle Berechenbarkeitsmodelle

Die Busy-Beaver-Funktion

Definition (Busy-Beaver-Funktion BB)

$BB(n)$ ist die maximale Zahl von 1-en, die eine TM mit n Zuständen und $\Gamma = \{B, 1\}$, angewendet auf das leere Band, bei Termination hinterlässt.

Theorem

BB ist unberechenbar (d.h., nicht Turing-berechenbar).

Anmerkungen:

- BB ist eine totale Funktion vom Typ $\mathbb{N} \to \mathbb{N}$.
- Man kennt die Werte von $BB(n)$ nur für $n = 1, 2, 3, 4, 5$.
- Würde man, z.B., $BB(100)$ kennen, so wären damit viele berühmte offene mathematische Probleme (im Prinzip) gelöst.
- Für $\Gamma = \{B, 0, 1\}$ ist bereits unbekannt was mit nur 3 Zuständen auf einer TM berechenbar ist. Bei 4 Bandsymbolen kann man nicht einmal mehr TMs mit nur 3 Zuständen vollständig analysieren!
- Siehe (z.B.) `https://en.wikipedia.org/wiki/Busy_beaver`

Kolmogorov-Komplexität: Perfekte Komprimierung

Definition (Kolmogorov-Komplexität $K(n)$)

$K(n)$ ist die minimale Länge des Binärcodes einer TM die, angewendet auf das leere Band, $[n]_2$ ausgibt.

Anmerkung: Die exakte Definition bezieht sich auf eine universelle TM.

Theorem

K ist eine (Turing-)unberechenbare Funktion (vom Typ $\mathbb{N} \to \mathbb{N}$).

Anmerkungen:

- Intuition: $K(n)$ gibt an, wie knapp man n eindeutig beschreiben kann.
- Das ist auch in der Praxis relevant:
  Stichworte: Optimale Codes, maximaler Informationsgehalt,...
- Trotz der Unberechenbarkeit lässt sich viel Allgemeines über K sagen:
  Z.B.: Es gibt ein c, sodass $K(n) \leq \log_2(n) + c$ für alle n,
  andererseits $K(n) > \log_2(n) - c$ für 'die allermeisten' n.

Das Postsche Korrespondenzproblem

- Gegeben: Endliche Menge von Wort-Paaren
  $$\{(v_i, w_i) \mid 1 \leq i \leq n,\ v_i, w_i \in \Sigma^*\}$$
  Frage: Gibt es eine Folge $i_1, ..., i_k$, wobei $i_\ell \in \{1, ..., n\}$ für $1 \leq \ell \leq k$, sodass $v_{i_1} ... v_{i_k} = w_{i_1} ... w_{i_k}$?

Theorem

Das Postsche Korrespondenzproblem ist unentscheidbar.

Beispiel:
Gegeben: $\{(1, 101), (10, 00), (011, 11)\}$, wobei
$v_1 = 1,\ v_2 = 10,\ v_3 = 011,\ \ w_1 = 101,\ w_2 = 00,\ w_3 = 11$
Eine Lösung: $1, 3, 2, 3$, da $1 \cdot 011 \cdot 10 \cdot 011 = 101 \cdot 11 \cdot 00 \cdot 11$

Anmerkungen:

- Die kleinste Lösung für $\{(001, 0), (01, 011), (01, 101), (10, 001)\}$ benötigt bereits eine Indexfolge der Länge 66 (also 66 Wort-Paare)!
- Für $\Sigma = \{1\}$ ist das Postsche Korrespondenzproblem noch entscheidbar; aber bereits unentscheidbar für $\Sigma = \{0, 1\}$

Hinweise zu anderen unentscheidbarer Probleme (als Anregung Details dazu selbst zu erforschen!)

- Wortproblem für (endlich erzeugte) Gruppen
- Tiling: Gegeben Quadrate mit verschiedenen Kantenfarben, kann man die gesamte Ebene so belegen, dass alle Kantenfarben passen?
- Conway's Game of Life: Sind bestimmte Konfigurationen erreichbar
- Zellular-Automaten: (Z.B.) Periodizität des Ablaufs?
- Ray-Tracing: Erreicht ein Lichtstrahl, der von einem bestimmten Punkt ausgeht und von den Oberflächen gegebener geometrischer Objekte reflektiert wird, ein bestimmtes Areal?
- Optimale Flugrouten unter Berücksichtigung realer Kostenregeln

  `http://www.demarcken.org/carl/papers/ITA-software-travel-complexity/ITA-software-travel-complexity.pdf`
- (viele Probleme aus Topologie, Analysis, Physik, Ökonomie, etc.)

Teil 2 – Berechenbarkeit (Überblick)

Teil 2.1: Die Church-Turing These

Teil 2.2: Imperatives Paradigma: Turingmaschinen und mehr

Teil 2.3: Funktionales Paradigma: λ-Kalkül

Teil 2.4: Rekursive Funktionen

Teil 2.5: Der Satz von Rice und seine Anwendung

Teil 2.6: Weitere unentscheidbare Probleme

Teil 2.7: Unkonventionelle Berechenbarkeitsmodelle

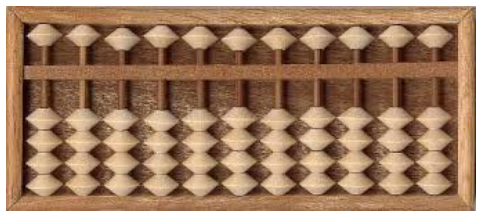
Mechanische Rechner

Mechanische Rechenhilfen und Rechenautomaten sind ein Thema vor allem der Geschichte, aber auch der Didaktik der Informatik.

Aus heutiger Sicht: Spezialanwendungen, keine Berechnungs-Universalität

Einige Beispiele:

- Abakus (Rechenbrett):



- Charles Babbage' Difference Engine (1820)†, Analytic Engine (1838)†:
  → Lady Ada Lovelace wird erste Programmiererin
- MONIAC† (Monetary National Income Analogue Computer): auch 'Financephalograph', erfunden 1949 vom neuseeländischen Ökonomen Bill Phillips, um die Wirtschaft Großbritanniens zu simulieren.
- Гидравлическиы интегратор† ('Wasser-Integrierer'): Vladimir S. Lukyanov (1936), zur Lösung inhomogener Differentialgleichungen

Andere physikalische und biochemische Grundlagen

Stichworte zu den beiden prominentesten Beispielen:

- Quantum Computing†
  - beruht auf Superposition und Verschränkung
  - inhärent probabilistisch: Berechnung relativ zu Fehlergrenzen
  - Formales Modell: QTM (Quantum-Turing-Maschine)
    modulo beliebig kleiner ϵ-Schranken kann auf einer QTM dasselbe berechnet und entschieden werden wie auf einer TM
  - bestimmte Probleme könnten exponentiell schneller gelöst werden
  - für NP-vollständige Probleme sind keine polynomiellen Quanten-algorithmen bekannt; man vermutet, dass solche nicht existieren
  - Komplexitätsgewinne wohl nur für Spezialanwendungen zu erwarten
  - in der Praxis: große Herausforderungen durch Dekohärenz

- DNA Computing†
  - nützt die komplexen biochemischen Prozesse der Genetik
  - (zumindest in der Theorie:) universell im Sinn der Church-Turing-These
  - massive Parallelität auf kleinstem Raum mit wenig Energie
  - Spezialfall des größeren Gebiets des 'Molekularen Rechnens'

Interaktives Rechnen

Viele aktuelle Anwendungs-Scenarios beruhen auf massiver Interaktion zwischen Rechner und Nutzer – allgemeiner: Umwelt.

Das stellt eine Herausforderung für die Berechenbarkeitstheorie, ohne die Church-Turing-These grundsätzlich zu verneinen.

Einige Aspekte aus aktueller Forschung und Praxis:

- Modelle der Kommunikation und Interaktion zwischen Agenten
- Rechner-Umwelt-Interaktion als (formales) Spiel ('Game Semantics')
- Verifikation offener Systeme
- massive, fehlerbehaftete, unsichere Web-Interaktionen

Leseempfehlung:
Dina Goldin, Scott A. Smolka, Peter Wegner (Herausgeber)
Interactive Computation – The New Paradigm, Springer 2006.